SpikingConvNeXt: TTFS Conversion of ConvNeXt with Minimum Residual Merging and Learnable Delays for Sparse Inference

<Author 1>, <Author 2>, ...

<Affiliation(s)>
<Corresponding author email>

Prepared for submission to <Target Journal Name> (Q1).

Abstract

Time-to-first-spike (TTFS) coding is an attractive paradigm for energy-efficient spiking neural networks (SNNs) because each neuron emits at most one spike, and more salient information is encoded by earlier spikes. Recent approximation-free TTFS theory and training methods can match deep ReLU accuracy while operating with high spiking sparsity [2]. In this work, we propose SpikingConvNeXt: a conversion of ConvNeXt [1] into a TTFS-SNN that (i) removes activation and normalization layers, (ii) replaces pointwise MLP transformations with an analytic spike-time mapping, and (iii) introduces a time-domain residual merge based on elementwise minimum rather than summation. The minimum residual merge is motivated by the interpretation of early spikes as high-saliency evidence: in the time domain, adding residual paths delays spikes and can suppress critical early information, while a minimum operator preserves the earliest arriving evidence. To control sparsity, we further introduce learnable per-output delays in the TTFS mapping and optionally constrain pointwise weights to be non-negative to bias neurons toward late or absent spikes. On CIFAR-10, SpikingConvNeXt-Tiny reaches 95.96% top-1 accuracy with 33.3% activation sparsity and achieves up to ~38% activation sparsity with a modest accuracy trade-off (94.43%). Results for CIFAR-100 and MNIST are left as placeholders to be completed.

Keywords: Spiking neural networks; time-to-first-spike; TTFS; ConvNeXt; sparsity; residual connections; neuromorphic computing

Contributions

- A TTFS-based spiking conversion of ConvNeXt blocks that removes GELU and normalization layers while reusing ConvNeXt weights.
- A time-domain residual merge operator based on elementwise minimum that improves information flow for spike-time representations.
- Learnable per-output delays (and optional non-negative pointwise weights) to directly steer activation sparsity in deep TTFS networks.
- A practical PyTorch implementation and a reproducible evaluation protocol on CIFAR-10/100 and MNIST (placeholders included).

1. Introduction and Related Work

Spiking neural networks (SNNs) promise event-driven computation where neurons communicate with sparse spikes rather than dense real-valued activations. Among SNN formulations, time-to-first-spike (TTFS) is attractive because it represents information with a single spike time per neuron inside a fixed window, enabling low-latency inference when early spikes carry sufficient evidence [2].

At the same time, modern ConvNet backbones such as ConvNeXt deliver strong accuracy with simple convolutional building blocks, but their standard design relies on dense activations and residual addition in the value domain [1]. Directly translating these components into TTFS is not straightforward: nonlinearities such as GELU and normalization layers do not naturally operate on spike times, and additive residual merges can blur the temporal semantics where earlier spikes should dominate.

This paper introduces SpikingConvNeXt, a TTFS variant of ConvNeXt that operates explicitly in the spike-time domain. We remove GELU and normalization layers and replace the pointwise MLP path with an analytic TTFS mapping adopted from prior TTFS work [2]. Each block propagates spike times through depthwise convolution on time maps followed by two TTFS pointwise projections implemented as closed-form updates.

Contributions. We make the following contributions:

- A ConvNeXt-to-TTFS block design that reuses ConvNeXt depthwise and pointwise weights while replacing value-domain nonlinearities with analytic spike-time updates.
- A time-domain residual fusion rule that replaces summation with an elementwise minimum over spike times, preserving the earliest evidence across residual and block paths.
- Learnable per-output delays (D_mid and D_out) and optional non-negative pointwise weights to steer spike timing toward later or absent spikes, increasing activation sparsity.
- An empirical study on CIFAR-10 with additional evaluation placeholders for CIFAR-100 and MNIST, reporting accuracy and sparsity metrics under the TTFS time window.

SpikingConvNeXt therefore bridges a high-performing ConvNet backbone [1] with an approximation-free TTFS computation model [2] and introduces minimum-based residual fusion as a simple, interpretable mechanism for preserving TTFS semantics while promoting sparsity.

2. Method

2.1. ConvNeXt backbone and modifications

ConvNeXt blocks consist of a depthwise convolution, followed by a pointwise MLP implemented as two 1×1 linear transformations with a nonlinearity (GELU) in between, and a residual connection [1]. In our implementation, we remove normalization layers and replace GELU with an analytic spike-time mapping inspired by the TTFS conversion framework [2]. We reuse the ConvNeXt weights for the depthwise and pointwise layers and wrap each block with a spike-time computation module.

Figure 1.

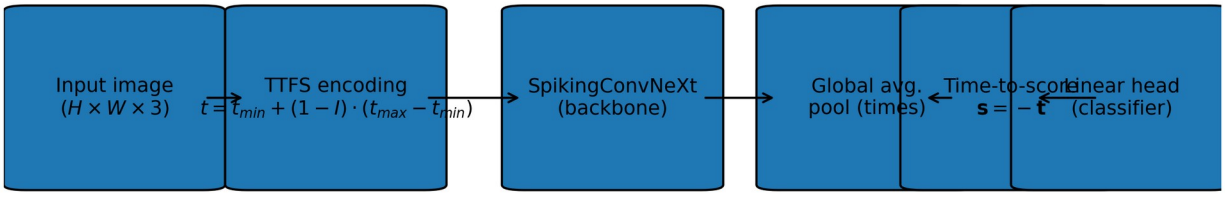

Figure 1. End-to-end pipeline of SpikingConvNeXt: input images are encoded into TTFS spike times, processed by a SpikingConvNeXt backbone, pooled, converted to scores, and mapped to logits.

Figure 2.

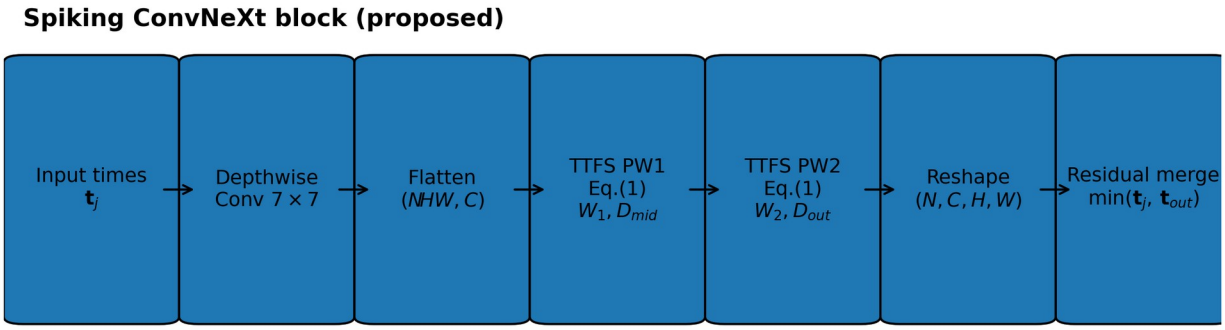

Figure 2. Proposed spiking variant of a ConvNeXt block. The depthwise convolution operates on spike-time maps, while the pointwise transforms are implemented by an analytic TTFS mapping with learnable delays. The residual merge uses an elementwise minimum operator.

2.2. TTFS encoding of images

Given an input image tensor I ∈ [0,1]^{C×H×W}, we encode each pixel intensity into a spike time within a fixed time window [t_min, t_max] using a linear TTFS encoding:

```
t = t_min + (1 − I) · (t_max − t_min).
```

Higher intensity values map to earlier spike times (closer to t_min), while low intensities map to later times (closer to t_max). In our implementation, a spike time clamped to t_max is treated as “no spike” for sparsity accounting.

2.3. Analytic spike-time mapping for pointwise layers

To replace the ANN pointwise MLP (Linear → GELU → Linear) with a TTFS operation, we compute output spike times directly from input spike times. For a vector of input spike times t_j, a weight matrix W, and a per-output delay D_i, we compute:

```
(1)  threshold = t_max − t_min − D_i
(2)  t_i = (t_j − t_min) · W + threshold + t_min
```

```
(3)  t_i = min(t_i, t_max)
```

This operation corresponds to the PyTorch function call_spiking_torch used in our implementation. Delays D_i are learned and constrained to be non-negative and bounded within the time window.

Algorithm 2. Analytic TTFS mapping used in pointwise layers.

Algorithm 2 CallSpiking: Analytic TTFS Spike-Time Mapping

Input: t_j ∈ R^{B×C_in} (input spike times), W ∈ R^{C_in×C_out} (weights), D ∈ R^{C_out} (delay), t_min, t_max

Output: t_i ∈ R^{B×C_out} (output spike times)

1: θ ← (t_max − t_min) − D // threshold term (Eq. 18 form)
2: Δ ← t_j − t_min
3: t_i ← (Δ · W) + θ + t_min
4: t_i ← where(t_i < t_max, t_i, t_max) // clamp late/non-spikes to t_max
5: return t_i

2.4. SpikingConvNeXt block

Each ConvNeXt block is converted into a SpikingBlock that reuses the original depthwise and pointwise weights. Given an input spike-time tensor t_j ∈ ℝ^{N×C×H×W}, we apply the depthwise convolution directly on the spike-time map as a practical spatial mixing heuristic. We then flatten spatial positions and apply the analytic TTFS mapping twice, corresponding to the two pointwise linear layers. We introduce two learnable delay vectors, D_mid and D_out, for the hidden and output pointwise transforms.

Algorithm 1. Spiking ConvNeXt block forward pass (TTFS mapping with min-residual).

Algorithm 1 Algorithm of Spiking ConvNeXt Block

Input: $t_j \in R^{N\times C\times H\times W}$ (input spike times), t_{min}, t_{max}; DWConv(·); W1, W2; D_mid, D_out; DropPath(·); force_positive

Output: $t_{out} \in R^{N\times C\times H\times W}$ (output spike times)

x ← DWConv(t_j)
(N, C, H, W) ← shape(x)
x_flat ← reshape(permute(x, N,H,W,C), (N·H·W, C))
if force_positive then
    W1 ← ReLU(W1); W2 ← ReLU(W2) // enforce non-negative pointwise weights
end if
D_mid ← clamp(ReLU(D_mid), 0, 0.9·(t_max − t_min)) // learnable per-output delays
t_mid ← CallSpiking(x_flat, W1^T, D_mid, t_min, t_max)
D_out ← clamp(ReLU(D_out), 0, 0.9·(t_max − t_min))
t_blk ← CallSpiking(t_mid, W2^T, D_out, t_min, t_max)
t_blk ← permute(reshape(t_blk, (N, H, W, C)), N,C,H,W)
t_out ← min(t_j, DropPath(t_blk)) // time-domain residual fusion
return t_out

Innovations in this block. Compared with the original ConvNeXt block, we (i) remove normalization and GELU and replace the pointwise MLP with an analytic TTFS mapping (Algorithm 2), (ii) fuse the residual and block output in the time domain using an elementwise minimum instead of summation so earlier spikes dominate, and (iii) introduce learnable non-negative delays (D_mid, D_out) and optionally non-negative pointwise weights to push spike times toward t_max, increasing silent-neuron rate.

2.5. Minimum residual merge in the time domain

In standard ConvNeXt (and most ANNs), the residual merge is additive: y = x + f(x). In a TTFS network, signals represent spike times rather than activation magnitudes, and addition does not preserve the semantics of latency. Moreover, adding spike times generally delays the resulting value, potentially suppressing early evidence. We therefore propose an elementwise minimum residual merge:

```
t_out = min(t_j, f_TTFS(t_j)).
```

Figure 3 contrasts the standard additive residual merge with the proposed minimum operator in the TTFS time domain; Figure 4 provides a numeric illustration.

This choice is consistent with TTFS semantics where earlier spike times indicate more salient evidence [2]. The minimum operator selects the earliest available evidence across the shortcut and transformed paths.

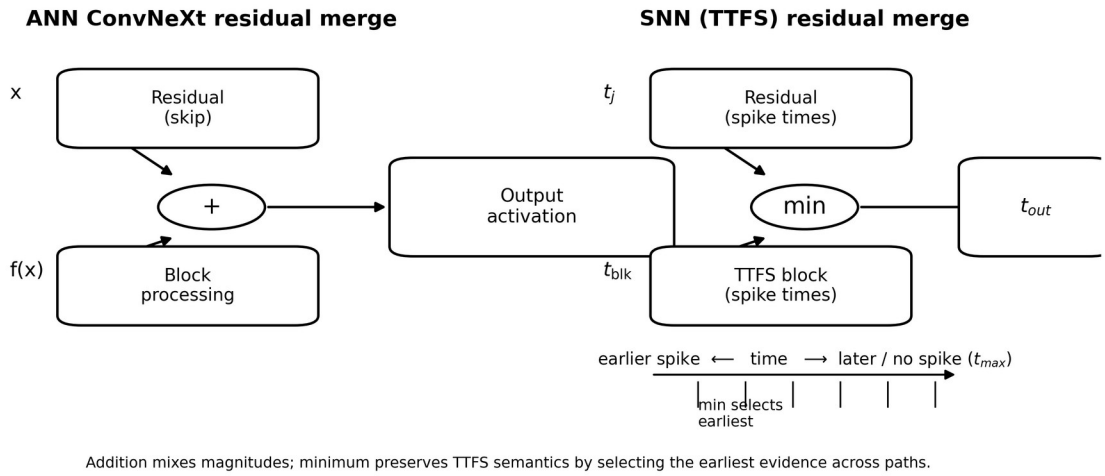

Figure 3. *Residual fusion in ConvNeXt (addition) versus SpikingConvNeXt in the TTFS spike-time domain (minimum). The minimum operator preserves earliest evidence; late or absent spikes are represented by clamping to t_max.*

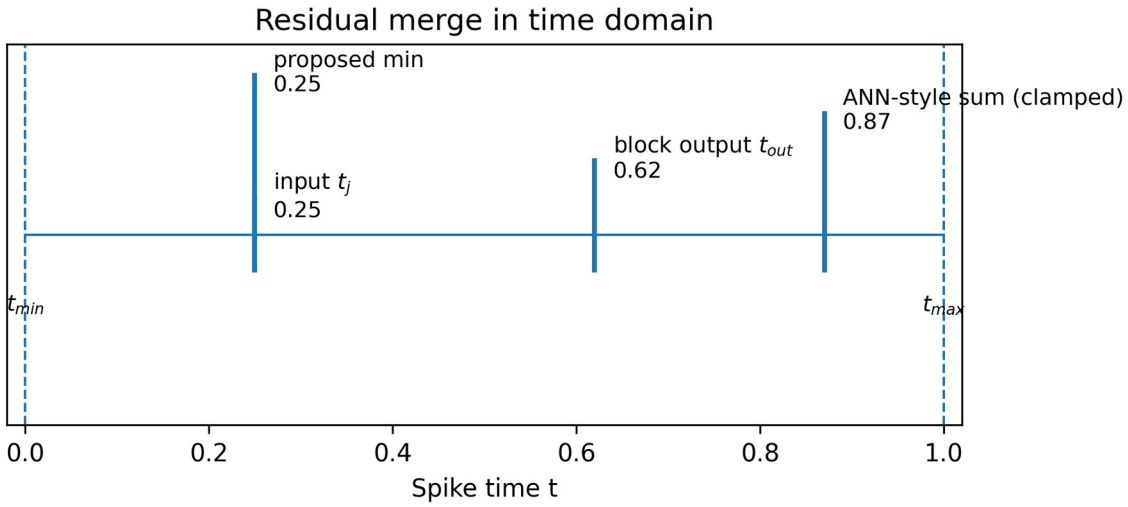

Figure 4. Illustration of residual merging in the time domain. ANN-style summation delays spike times, while the proposed minimum merge preserves the earliest (most salient) spike.

2.6. Sparsity mechanisms

We incorporate two mechanisms to steer activation sparsity in deep TTFS networks:

- Learnable delays: D_mid and D_out are trained to control the effective timing threshold per output neuron.
- Optional non-negative weights: we optionally enforce W1 and W2 to be non-negative (via ReLU on weights) to bias outputs toward late spikes and increase the fraction of neurons clamped at t_max.

2.7. Why non-negative pointwise weights can increase sparsity

Let $\Delta_j = (t_j - t_{min}) \geq 0$ denote the time offset from the window start. From Eq. (2), the pre-clamp output is $t_i = \Delta_j \cdot W + (t_{max} - t_{min} - D_i) + t_{min}$. When $\Delta_j \geq 0$ and $W \geq 0$, the product term $\Delta_j \cdot W$ is non-negative, which shifts t_i toward larger values (later spikes). More values therefore exceed t_{max} and are clamped to t_{max}, which we count as silent (no spike) events. This provides an intuitive mechanism to bias the network toward sparse spiking activity, at the cost of reduced representational flexibility if enforced too strongly.

Activation sparsity is measured as the fraction of neurons that do not fire within the time window, operationalized as spike times equal to t_{max} (after clamping). If T is the set of all spike-time values in a layer for a batch, sparsity is computed as:

$$S = |\{t \in T : t = t_{max}\}| / |T|.$$

3. Experiments

3.1. Datasets

We evaluate on CIFAR-10, CIFAR-100, and MNIST. CIFAR-10 and CIFAR-100 are 32×32 natural image datasets with 10 and 100 classes, respectively. MNIST is a 28×28 grayscale digit dataset with 10 classes. All inputs are scaled to [0,1] and encoded into spike times using the TTFS encoder in Section 3.2.

3.2. Model variants

Unless otherwise stated, we use ConvNeXt-Tiny as the backbone configuration (depths = (3, 3, 9, 3), dims = (96, 192, 384, 768)) [1]. We consider the following variants:

- Try 1 (baseline conversion): Replace the GELU pointwise nonlinearity with TTFS mapping; keep residual merge behavior unchanged.
- Try 2 (minimum residual): Use the proposed elementwise minimum residual merge; delay terms fixed to $D = 0$.
- Try 3 (learnable delays): Use minimum residual merge and enable learning of D_{mid} and D_{out} to increase activation sparsity.

3.3. Training and evaluation protocol

Training details are provided as placeholders to be completed:
• Initialization: <trained from scratch / initialized from pretrained ConvNeXt weights>
• Optimizer: <e.g., SGD / AdamW>, learning rate: <...>, weight decay: <...>
• Augmentation: <random crop, flip, cutout, mixup, ...>
• Epochs: <...>, batch size: <...>, hardware: <GPU/TPU details>
During evaluation, we use a fixed time window $t_{min} = 0.0$ and $t_{max} = 1.0$ and report top-1 accuracy and activation sparsity.

4. Results

4.1. CIFAR-10 results

Table 1 summarizes CIFAR-10 test results from our experiments. The proposed minimum residual merge (Try 2) improves accuracy over the baseline spiking conversion (Try 1). Enabling learnable delays (Try 3) increases activation sparsity to approximately 38% with a modest accuracy reduction.

Table 1. Test accuracy and activation sparsity (CIFAR-10 filled; CIFAR-100 and MNIST left as placeholders).

Method	CIFAR-10 Acc. (%)	CIFAR-10 Sparsity (%)	CIFAR-100 Acc. (%)	CIFAR-100 Sparsity (%)	MNIST Acc. (%)	MNIST Sparsity (%)
Nazari et al. [placeholder ref]	93.60	-	<TBD>	<TBD>	<TBD>	<TBD>
Simonyan (ANN) [placeholder ref]	93.56	0 (?)	<TBD>	<TBD>	<TBD>	<TBD>
Stanojevic et al. (TTFS baseline) [2]	93.59	38.00	<TBD>	<TBD>	<TBD>	<TBD>
Ours (SNN) Try 1	95.29	33.30	<TBD>	<TBD>	<TBD>	<TBD>
Ours (SNN) Try 2 (min residual)	95.96	33.30	<TBD>	<TBD>	<TBD>	<TBD>
Ours (SNN) Try 3-A (learnable delays)	94.11	38.18	<TBD>	<TBD>	<TBD>	<TBD>
Ours (SNN) Try 3-B (learnable delays)	94.43	38.01	<TBD>	<TBD>	<TBD>	<TBD>
Ours (SNN) Try 3-C (learnable delays)	94.05	38.50	<TBD>	<TBD>	<TBD>	<TBD>

Figure 4.

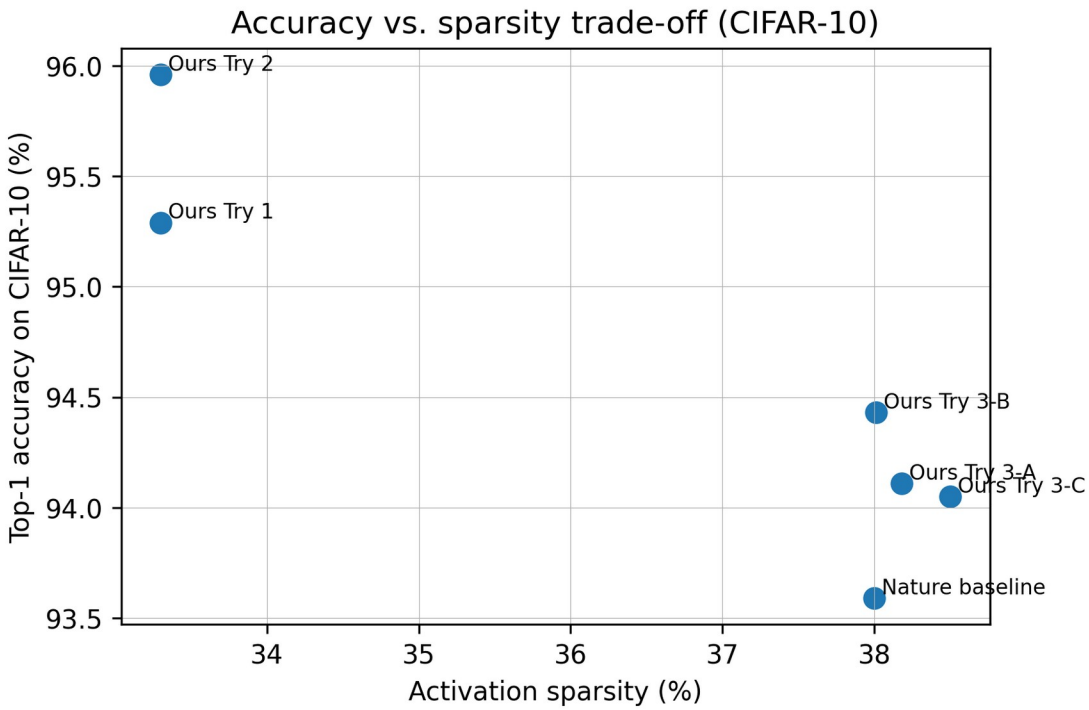

Figure 4. Accuracy vs. sparsity trade-off on CIFAR-10. Minimum-residual variants achieve higher accuracy at comparable sparsity, while learnable delays push sparsity upward with a modest accuracy trade-off.

Figure 5.

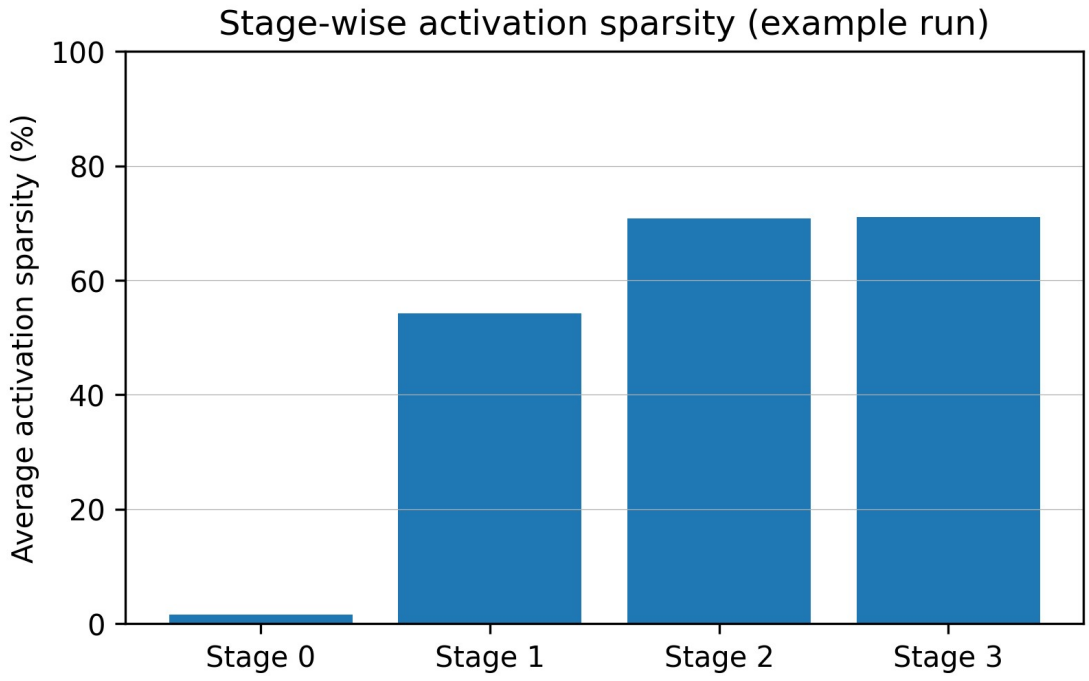

Figure 5. Example stage-wise activation sparsity (Try 3-B snapshot). Early stages exhibit low sparsity compared to later stages, highlighting opportunities to further sparsify early feature extraction.

4.2. Discussion

Why does minimum residual help? In TTFS coding, an earlier spike implies a larger effective activation (higher saliency) [2]. In an additive residual, combining spike times with summation produces larger latencies, which can suppress early evidence. The minimum residual merge avoids this artificial delay and acts as a simple deterministic rule to preserve early evidence.

Sparsity control. Learnable delays increase activation sparsity by allowing the model to shift some neurons toward late spikes that are clamped to t_max, effectively silencing them. The observed stage-wise sparsity suggests that later stages are easier to sparsify than early stages; future work can explore stage-specific delays or additional regularizers to increase sparsity in early layers without harming accuracy.

5. Limitations and Future Work

This manuscript is a draft intended for journal submission and contains placeholders that must be completed (training details, CIFAR-100 and MNIST results, statistical significance over multiple runs, and baseline citations).

Methodologically, the depthwise convolution is applied directly to spike-time maps as a practical heuristic. A more principled TTFS-compatible convolution operator may further improve accuracy and sparsity. Additionally, enforcing strictly non-negative pointwise weights may increase sparsity but can limit representational capacity; the best constraint likely depends on the dataset and should be explored with ablations.

For a Q1 submission, we recommend: (i) report mean±std over ≥3 seeds, (ii) provide energy/latency proxies (spikes/neuron, silent ratio, and optionally FLOPs or hardware measurements), (iii) compare against strong modern SNN baselines and ANN ConvNeXt, and (iv) include an ablation over t_min/t_max and delay initialization.

6. Conclusion

We introduced SpikingConvNeXt, a TTFS-based conversion of ConvNeXt [1] that removes activations and normalization, replaces pointwise MLP computations with an analytic spike-time mapping, and proposes a minimum residual merge suited to time-domain representations. The approach achieves strong CIFAR-10 performance (up to 95.96% top-1) while enabling controllable sparsity through learnable delays. These results suggest that modern ConvNet backbones can be adapted to TTFS SNNs with minimal structural changes and simple time-domain operators, paving the way for energy-efficient deployment on neuromorphic and event-driven hardware.

Acknowledgments

<Acknowledgments text (funding, compute grants, collaborators, etc.)>

Data Availability

All datasets used are publicly available (CIFAR-10/100 and MNIST). Exact preprocessing and splits: <provide details / repository link>.

Code Availability

Source code and trained checkpoints will be released at: <GitHub/Zenodo link> upon publication.

Appendix A: Key Implementation Snippets

This appendix includes short code snippets for clarity. For full source code, see the repository link in Code Availability.

Listing A1: Analytic TTFS mapping (call_spiking_torch).

```
def call_spiking_torch(tj, W, D_i, t_min_prev, t_min, t_max):
    # threshold (Eq. 18 in the reference TTFS formulation)
    threshold = t_max - t_min - D_i
    tj = tj.to(W.dtype)
    threshold = threshold.to(W.dtype)
    delta = tj - t_min
    ti = torch.matmul(delta, W) + threshold + t_min
    ti = torch.where(ti < t_max, ti, t_max)
    return ti
```

Listing A2: Time-domain residual merge.

```
# ANN residual:
# out = tj + self.drop_path(t_out)

# Proposed TTFS residual merge:
out = torch.minimum(tj, self.drop_path(t_out))
```

References

[1] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. A ConvNet for the 2020s (ConvNeXt). arXiv:2201.03545, 2022.

[2] Stanojevic, A., Gehring, W., Woźniak, S., Binas, J., Chicca, E., & Payeur, A. High-performance deep spiking neural networks with 0.3 spikes per neuron. Nature Communications, 15:6793, 2024.

[3] Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report (CIFAR), 2009.

[4] LeCun, Y., Cortes, C., & Burges, C. J. C. The MNIST database of handwritten digits, 1998.

[5] <Add additional SNN/TTFS references used in Related Work and baselines.>